

JAVA TCP CHAT APPLICATION

Project Report

Project Title: Java TCP Chat Application

Technology: Java

Communication Protocol: TCP

Architecture: Client-Server

Submitted in partial fulfillment of the requirements for the completion of
the course

College Project

1. Abstract

The Java TCP Chat Application is a client-server based messaging system developed using Java networking and multithreading concepts. The application allows multiple users to connect to a central server and communicate through chat rooms.

The system uses TCP sockets to establish reliable communication between clients and the server. The server accepts multiple client connections and creates a separate thread for each connected client. A `ClientHandler` manages the communication and activities of each individual user.

The application supports unique username registration, public chat rooms, private messaging, viewing available rooms, viewing users in the current room, joining different rooms, timestamps, and graceful disconnection. The project demonstrates Socket Programming, `ServerSocket`, Multithreading, Collections, Concurrent Collections, Exception Handling, Lambda Expressions, and Input/Output Streams.

2. Introduction

Communication applications are an important example of computer networking. This project implements a simple chat system using Java's built-in networking libraries and follows the client-server model, where the server acts as the central communication point and clients connect to it to exchange messages.

The project consists of three main Java classes: `ChatServer`, `ClientHandler`, and `ChatClient`. `ChatServer` listens for incoming connections. Whenever a client connects, the server creates a `ClientHandler` and assigns it to a separate thread. The `ClientHandler` manages that client's communication, username, room membership, commands, and disconnection. `ChatClient` provides the interface through which users connect, send messages, and receive messages.

3. Problem Statement

The objective is to develop a Java-based chat application that establishes TCP connections, supports multiple clients, assigns unique usernames, provides public chat rooms, supports private messages, displays timestamps, and handles client disconnections properly.

4. Objectives

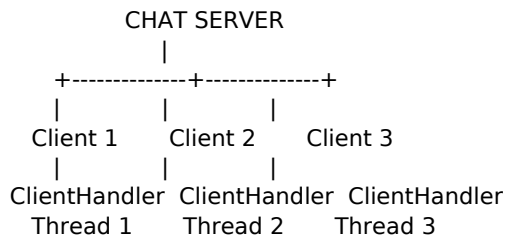
- Understand client-server architecture.
- Implement network communication using TCP sockets.
- Understand Java `Socket` and `ServerSocket` classes.
- Implement multithreading for multiple simultaneous clients.
- Implement public and private messaging.
- Manage users and chat rooms using Java collections.
- Understand thread-safe data structures.
- Implement command-based interaction.
- Handle network errors and client disconnections.
- Develop a practical application using Java networking concepts.

5. Technologies Used

Technology	Purpose
Java	Main programming language
TCP	Reliable network communication
Socket	Client-server connection
ServerSocket	Listening for client connections
Multithreading	Handling multiple clients
BufferedReader	Reading incoming text
PrintWriter	Sending text
ConcurrentHashMap	Thread-safe client/room management
CopyOnWriteArraySet	Thread-safe room membership
Command Prompt / Terminal	Running the application

6. System Architecture

The project follows a client-server architecture. The server acts as the central communication point. Each connected client is assigned its own `ClientHandler` and thread.



7. Project Components

7.1 ChatServer

`ChatServer` starts the server, opens the required port, accepts client connections, maintains connected users and chat rooms, registers usernames, removes disconnected users, and broadcasts messages to room members. The default port is 5000.

7.2 ClientHandler

`ClientHandler` manages one connected client. It receives the username, checks uniqueness, joins the default room, reads client commands, processes public and private messages, switches rooms, displays users and rooms, handles `/quit`, and performs disconnection cleanup. Each client is handled by a separate thread.

7.3 ChatClient

`ChatClient` connects to the server, accepts server messages, reads keyboard input, sends messages, listens for incoming messages simultaneously, and supports the `/quit` command. Default connection values are localhost and port 5000.

```
java ChatClient localhost 5000
```

8. Working of the System

Step 1: Start the Server

The server is started first and listens on port 5000.

Step 2: Client Connects

The client creates a socket connection to the server. The server accepts it using `ServerSocket.accept()`.

Step 3: ClientHandler is Created

After accepting a connection, the server creates a `ClientHandler` and starts a new thread.

Step 4: Username Selection

The client is asked for a username. The server checks whether it is unique.

Step 5: Joining the Default Room

After successful registration, the user is automatically placed in the general room.

Step 6: Sending Public Messages

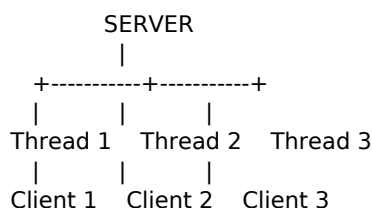
Normal messages are sent to the server and broadcast to members of the current room.

Step 7: Receiving Messages

The client listener thread continuously waits for messages from the server while the main thread handles keyboard input.

9. Multithreading

Multithreading is a key concept in this project. Each connected client receives an independent `ClientHandler` and thread. This prevents one client from blocking communication with other clients.



The client also uses a listener thread so it can receive server messages while its main thread waits for keyboard input.

10. Chat Commands

Command	Function
<code>/join <room></code>	Join or create a chat room
<code>/rooms</code>	Display available rooms
<code>/users</code>	Display users in the current room
<code>/msg <user> <message></code>	Send a private message

Command	Function
/quit	Disconnect from the server

11. Chat Rooms

The server maintains rooms using a map in the form Room Name → Set of Clients. For example, general can contain User1 and User2, while football can contain User3 and User4.

Users can switch rooms using /join football. When a user leaves a room, the server removes them from that room. Empty custom rooms are removed, while the general room is kept permanently.

12. Private Messaging

Private messaging is supported using the /msg command. The server finds the target user through the clients map and sends the message directly to that user's ClientHandler.

```
/msg Rahul Hello Rahul!
```

The target receives a message such as: [18:40:10] [PM from Anubhav]: Hello Rahul!. The sender also receives a confirmation showing that the message was sent.

13. Timestamp System

Messages contain timestamps using DateFormatter with the HH:mm:ss pattern. A typical public message appears as:

```
[18:42:35] [general] User1: Hello!
```

14. Thread-Safe Data Structures

ConcurrentHashMap

ConcurrentHashMap is used for the clients map and rooms map. It is appropriate for shared information accessed by multiple client-handler threads.

CopyOnWriteArraySet

CopyOnWriteArraySet is used for room membership. It supports safe concurrent access when different client threads add or remove members.

15. Important Java Concepts Demonstrated

- Object-Oriented Programming: Separate classes are used for the server, client handler, and client.
- Interfaces: ClientHandler implements Runnable so it can be executed by a thread.
- Multithreading: Separate threads handle connected clients.
- Exception Handling: try, catch, and finally handle network errors and cleanup.
- Collections: Map and Set store users and rooms.
- Lambda Expression: ChatClient uses a lambda expression for its listener thread.
- Try-With-Resources: Network resources are automatically closed.

16. Algorithm

Server Algorithm

```
START
|
Set default port
|
Create ChatServer
|
Create ServerSocket
|
Create default "general" room
|
Wait for client
|
Client connects
|
Create ClientHandler
|
Create new thread
|
Handle client
|
Wait for next client
|
Repeat
```

ClientHandler Algorithm

```
START
|
Create input/output streams
|
Ask for username
|
Check username
|
Is username unique?
|---- NO ----> Ask again
|
YES
|
Register user
|
Join "general"
|
Read commands
|
+-----+-----+-----+
|   |   |   |   |
| Message Command /quit Invalid
|   |   |   |   |
| Broadcast Process Exit Error
|   command
|
Continue
```

17. Testing

Test Case	Expected Result
Server startup	Server listens on port 5000 and displays ready message.

Test Case	Expected Result
Client connection	Client displays successful connection message.
Unique username	User is registered and joins general.
Duplicate username	Server asks the user to choose another username.
Public message	Message is displayed to users in the same room.
Room switching	User leaves the old room and joins the new room.
Private message	Only target and sender receive the private-message exchange.
/users	Users in the current room are displayed.
/rooms	Available rooms are displayed.
/quit	Client disconnects and server cleans up its state.

18. Advantages

- Supports multiple clients.
- Uses reliable TCP communication.
- Provides public chat rooms.
- Supports private messaging.
- Supports unique usernames.
- Provides timestamps.
- Uses multithreading.
- Uses thread-safe collections.
- Handles client disconnection.
- Demonstrates practical Java networking.

19. Limitations

- Primarily a command-line application.
- No graphical user interface.
- Messages are not stored permanently in a database.
- No user authentication or password system.
- Communication is not encrypted.
- Server must remain running for clients to communicate.
- No persistent chat history after the server stops.

20. Future Scope

- Add a JavaFX or Swing graphical user interface.
- Integrate a database for user accounts and chat history.
- Add authentication with usernames and passwords.

- Implement SSL/TLS for encrypted communication.
- Add file and image sharing.
- Store and retrieve room message history.
- Add online/offline user status.

21. Learning Outcomes

- Understanding of client-server architecture.
- Practical experience with TCP Socket and ServerSocket programming.
- Understanding of multithreading and concurrent client handling.
- Experience with Java I/O using BufferedReader, InputStreamReader, and PrintWriter.
- Understanding of ConcurrentHashMap and CopyOnWriteArraySet.
- Experience with exception handling for network operations.
- Understanding of command processing.
- Experience with modular object-oriented program design.
- Understanding of real-time message communication.
- Practical networking experience through a working Java application.

22. Conclusion

The Java TCP Chat Application successfully demonstrates the implementation of a multi-client communication system using Java networking. The project uses TCP sockets for reliable communication and multithreading to handle multiple clients simultaneously.

ChatServer manages connections, users, and rooms; ClientHandler manages the activities of individual clients; and ChatClient allows users to connect, send messages, and receive messages.

The project also demonstrates object-oriented programming, networking, multithreading, collections, concurrent programming, exception handling, and input/output streams. It provides a practical foundation for advanced features such as graphical interfaces, authentication, encryption, databases, file sharing, and persistent messaging.

23. References

- Java Documentation — Networking and Socket Programming
- Java Documentation — Multithreading and Runnable
- Java Documentation — Collections Framework
- Java Documentation — Concurrent Collections
- Computer Networking concepts related to TCP/IP and Client-Server Architecture

24. Project File Structure

```
Chat Application
|
+-- ChatServer.java
```

```
|  
+-- ClientHandler.java  
|  
+-- ChatClient.java
```

The three files work together to create the complete chat application. ChatServer accepts connections, ClientHandler manages individual clients, and ChatClient provides the user-side interface.